CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

Genetic Algorithm-Based CNN Pruning and Knowledge Distillation Using Soft Computing Techniques

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

SOFT COMPUTING (22CAE01N)

CASE STUDY REPORT

Submitted By:

160124748093 – Bheeshma Shankar A

CASE STUDY GUIDE

Dr. Y. C. A. Padmanabha Reddy
Associate Professor

Department of CSE Artificial Intelligence and Machine Learning

CBIT, Hyderabad

Academic Year 2026–2027

Certificate

This is to certify that the case study report entitled **"Genetic Algorithm-Based CNN Pruning and Knowledge Distillation Using Soft Computing Techniques"** has been successfully carried out by the student during the academic year 2026–2027 under my guidance and supervision. The work presented in this report is a record of the student's sincere efforts and academic study.

160124748093 – Bheeshma Shankar A

Case Study Guide:
Dr. Y. C. A. Padmanabha Reddy
Associate Professor
Department of CSE AI&ML
CBIT, Hyderabad

Acknowledgement

I express my sincere gratitude to my case study guide, Dr. Y. C. A. Padmanabha Reddy, Associate Professor, Department of CSE (Artificial Intelligence and Machine Learning), CBIT, for his valuable guidance, continuous support, and encouragement throughout the completion of this Soft Computing case study. His suggestions and explanations helped me understand the concepts of genetic algorithms, structured CNN pruning, and knowledge distillation, and apply soft computing techniques to the selected problem.

I would also like to express my gratitude to Dr. Y. Ramadevi, Head of the Department, Department of CSE (AI & ML), for providing the necessary resources and a supportive academic environment that encouraged me to complete this case study successfully. I am also thankful to Chaitanya Bharathi Institute of Technology (CBIT) for providing the academic facilities and learning opportunities.

Finally, I am grateful to my family and friends for their constant support, encouragement, and motivation throughout the completion of this case study.

A. Bheeshma Shankar - 160124748093

Abstract

Deep convolutional neural networks (CNNs) give strong accuracy, but their large parameter counts and heavy computation make them hard to run on mobile and embedded devices. This case study analyses the paper by Chou et al. [1], which compresses CNNs by structured channel pruning guided by a Genetic Algorithm (GA) and then retrains the pruned network with Knowledge Distillation (RKD). The GA encodes one pruning ratio per layer as a chromosome and searches for a configuration that balances accuracy, parameters and floating-point operations. The authors report that, for the largest classifier of a gesture-recognition model on EgoGesture, parameters fall by 91.02% and FLOPs by 90.96% with a 1.82 percentage-point accuracy drop, and inference time falls from 3.6 ms to 2.1 ms; on ResNets, RKD matches label-only retraining in 150 instead of 200 epochs. The report restates the method's mathematics, checks the reported numbers, and critically evaluates the work. It also documents the student's first experimental step: a baseline CNN with 620,810 parameters trained on CIFAR-10 in PyTorch on a Tesla T4 GPU, which reached 79.44% validation accuracy. GA search, pruning and distillation on CIFAR-10 have not yet been run, so no compression results are claimed for them.

Keywords: Genetic algorithm, structured channel pruning, knowledge distillation, model compression, soft computing, CIFAR-10.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

List of Figures

Figure	Title	Page No.
2.1	Structured versus unstructured pruning (conceptual illustration).	5
4.1	Overall hybrid framework, from the baseline CNN to the compared compressed model.	11
4.2	Genetic Algorithm workflow for searching the pruning vector.	13
4.3	Teacher–student knowledge distillation used for retraining (RKD).	14
4.4	Computed curves of (a) the initial pruning ratio and (b) the temperature schedule.	14
5.1	Share of parameters and MACs by layer for the baseline CNN.	17
5.2	Proposed PyTorch implementation architecture (design only).	18
5.3	Proposed experimental evaluation (plan only; not yet executed).	18
6.1	Reported results for classifiers A to D: (a) accuracy, (b) parameter and FLOP reduction, (c) inference time.	21
6.2	Accuracy drop of LRW (200 epochs) and the proposed RKD (150 epochs).	22
6.3	Training loss and accuracy of the baseline CNN over eight epochs.	23

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

List of Tables

Table	Title	Page No.
2.1	Comparison of the four approaches (the Basis column separates the paper's statements from general comparisons)	4
2.2	Unstructured versus structured pruning	5
2.3	Illustration of GA operators on a three-layer chromosome (hypothetical numbers, not from the paper)	6
3.1	Software and hardware environment of the notebook (Notebook)	8
3.2	Data used in the reference paper and in this study	8
4.1	The ten stages of the hybrid methodology (Paper, Sec. 3)	10
4.2	Symbols and their values (Paper, Sec. 3–4)	11
5.1	Layer-wise parameters and MACs of the baseline CNN (parameters: Notebook; MACs: derived)	16
5.2	Training configuration of the baseline CNN (Notebook)	17
5.3	Settings of the reference paper compared with the plan for this study	18
6.1	Original versus pruned R(2+1)D-18 classifiers on EgoGesture (Paper, Table 1)	20
6.2	Pruned ResNets compared with two earlier methods (Paper, Table 2)	21
6.3	Retraining with RKD versus LRW [6] (Paper, Table 3)	22
6.4	Training log of the baseline CNN on CIFAR-10 (Notebook)	22
6.5	Status of the planned five-model comparison (M1 from the notebook; M2 to M5 Proposed)	23
6.6	Advantages and limitations of the hybrid approach	25
7.1	Next steps for completing the experiment	26

స్వయం తేజస్వినా భవ
1979

Table of Contents

S.No	Title	Page No.
1.	**1. Introduction**	**1**
	1.1 Background and Significance	1
	1.2 The Role of Soft Computing in Model Compression	1
	1.3 Key Challenges in CNN Compression	1
	1.4 Motivation and Report Structure	1
2.	**2. Literature Review and Theoretical Study**	**3**
	2.1 Evolution of Model Compression	3
	2.1.1 Unstructured and Structured Pruning	3
	2.1.2 Retraining after Pruning	3
	2.1.3 Knowledge Distillation	3
	2.2 Analysis of the Reference Paper	3
	2.3 Theoretical Background	4
	2.3.1 Convolutional Neural Networks	4
	2.3.2 Structured Channel Pruning	5
	2.3.3 Genetic Algorithm	5
	2.3.4 Knowledge Distillation	6
	2.4 Synthesis and Research Gap	6
3.	**3. System Requirements and Specifications**	**8**
	3.1 Hardware and Software Requirements	8
	3.1.1 Environment of the Reference Paper	8
	3.1.2 Environment of this Study	8
	3.2 Dataset Description and Characteristics	8
	3.3 Evaluation Metrics	9
4.	**4. Methodology and System Design**	**10**
	4.1 Methodology Workflow	10
	4.2 Proposed Hybrid Pipeline: Mathematical Formulation	11
	4.2.1 Chromosome and Initialisation	12
	4.2.2 Crossover and Mutation	12
	4.2.3 Fitness Function and Selection	13
	4.2.4 Structured Channel Pruning	13
	4.2.5 Retraining with Knowledge Distillation (RKD)	13
	4.3 Algorithm	15
5.	**5. Implementation Details and Experimental Setup**	**16**

S.No	Title	Page No.
	5.1 Implementation Environment	16
	5.2 Baseline CNN (Teacher) Architecture	16
	5.3 Training Protocol	17
	5.4 Channel-Importance Step	17
	5.5 Planned Experimental Design	17
6.	**6. Results and Performance Analysis**	**20**
	6.1 Overview	20
	6.2 Results Reported in the Reference Paper	20
	6.2.1 R(2+1)D-18 on EgoGesture	20
	6.2.2 ResNet-32 and ResNet-56 on CIFAR-10 and CIFAR-100	21
	6.3 Results from the Baseline Experiment of this Study	22
	6.4 Theoretical Interpretation of Results	23
	6.4.1 Percentage Reduction versus Percentage-Point Change	23
	6.4.2 Accuracy–Efficiency Trade-off	24
	6.4.3 Fewer FLOPs Do Not Guarantee Proportional Speed	24
	6.4.4 Observations on the Baseline	24
	6.5 Critical Evaluation	24
	6.6 Applications, Advantages and Limitations	25
	6.7 Summary of Findings	25
7.	**7. Conclusion and Future Scope**	**26**
	7.1 Conclusion	26
	7.2 Limitations	26
	7.3 Roadmap and Future Scope	26
8.	**8. Appendix I: Notebook Summary, Glossary and Verification**	**27**
	8.1 Notebook Cells and Status	27
	8.2 Glossary	27
	8.3 Numerical Verification Log	27
9.	**9. References**	**29**

INSTITUTE OF TECHNOLOGY

1. Introduction

1.1 Background and Significance

A Convolutional Neural Network (CNN) learns visual patterns with small shared filters that slide over an image or a video frame. Early layers pick up edges and textures, and deeper layers combine them into parts and whole objects [7]. CNNs now power image recognition, object detection and gesture recognition.

This accuracy has a price. Every convolutional layer stores one filter for each output channel, and a deep network stacks many layers, so the parameter count quickly reaches millions. The largest classifier in the reference paper has 31.34 M parameters and needs 40.52 G floating-point operations (|FLOP|) [1]. AR/VR headsets and mobile phones have limited memory, battery and time budgets, so the authors argue that model size and computation must come down before such models can run in real time on the device [1].

The reference paper studied in this case study, by Chou et al. [1], proposes a hybrid compression method. A Genetic Algorithm (GA) chooses how many channels to prune in each layer, the channels are removed, and the pruned network is retrained with Knowledge Distillation (KD). This report analyses that paper, restates its mathematics, reviews its reported results, and documents the first experimental step carried out by the student: training the baseline CNN on CIFAR-10.

1.2 The Role of Soft Computing in Model Compression

Soft Computing covers methods that find good, not necessarily perfect, answers to hard problems, such as evolutionary algorithms, neural networks and fuzzy systems. A Genetic Algorithm is an evolutionary method: it keeps a population of candidate solutions and improves them through selection, crossover and mutation, without gradients and without trying every option. Choosing a pruning ratio for each layer is a search over a very large space; even ten possible ratios per layer give 10^N combinations for N layers (*analysis*). The quality of a configuration (accuracy, size, FLOPs) is not a smooth, differentiable function of the ratios (*analysis*), which makes a heuristic population-based search a natural fit.

1.3 Key Challenges in CNN Compression

- **Search space:** the number of possible per-layer pruning configurations is huge, so manual tuning is slow and trial-and-error.
- **Accuracy loss:** removing channels damages the network, and the damage must be repaired by retraining.
- **Retraining cost:** label-only retraining methods such as fine-tuning and learning-rate rewinding need many epochs [6].
- **Real speed-up:** fewer parameters do not always mean faster inference. In the paper, unstructured pruning removed 80% of the parameters but gave almost no CPU speed-up on a HoloLens [1].
- **Reproducibility:** the paper leaves several settings unspecified (population size, selection and mutation rates, training details), which makes exact reproduction difficult.

1.4 Motivation and Report Structure

The motivation of this case study is to understand, check and begin to reproduce a method that shrinks a CNN by about 90% without a large loss in accuracy. To keep sources clear, the report uses four labels: **(Paper)** for settings and results reported in [1]; **(Notebook)** for numbers printed by the student's own Colab notebook; *analysis* for interpretation or values derived from those numbers; and **Proposed** for work that has not been run. Where [1] gives no detail, the report says "Not specified in the reference paper."

Chapter 2 reviews the literature and the theory, and analyses the reference paper. Chapter 3 lists the requirements, data and evaluation metrics. Chapter 4 describes the methodology and its mathematics. Chapter 5 gives the implementation details and the experimental setup, including the baseline CNN. Chapter 6 presents and interprets the results. Chapter 7 concludes with limitations and a roadmap, Chapter 8 (Appendix I) collects supporting material, and Chapter 9 lists the references.

CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

2. Literature Review and Theoretical Study

2.1 Evolution of Model Compression

2.1.1 Unstructured and Structured Pruning

Pruning removes parts of a trained network that contribute little. Unstructured pruning sets individual small weights to zero [3]. It reduces the number of non-zero parameters, but the zeros sit at arbitrary positions, so special hardware or libraries are needed to turn them into real savings. Structured pruning removes whole filters or channels [4] and keeps the network regular, which is why it is preferred in practice. The importance of a channel can be scored with L1 or L2 norms, or with a Taylor expansion of the loss [5], which the reference paper uses [1].

2.1.2 Retraining after Pruning

Pruning lowers accuracy, so the network is retrained. Fine-Tuning (FT) retrains with a small learning rate, and Learning Rate Rewinding (LRW) replays part of the original learning-rate schedule and is reported to recover accuracy better [6]. Both minimise only the loss against the labels.

2.1.3 Knowledge Distillation

Knowledge Distillation lets a large teacher network guide a smaller student by passing on its softened output probabilities, not only the hard labels [2]. This gives the student extra information about how classes relate to each other, and it is the idea the reference paper adds to the retraining step.

2.2 Analysis of the Reference Paper

The reference paper is by K.-L. Chou, C.-L. Wang, Y.-C. Chen, W. Tang, Y.-T. Li, S.-C. Chang and C.-Y. Wang, published at the Great Lakes Symposium on VLSI (GLSVLSI ’25) [1]. It asks how to reduce a CNN’s size and computation for resource-constrained devices while keeping its accuracy. The authors contrast unstructured and structured pruning, show with a HoloLens measurement that unstructured pruning gave almost no CPU speed-up (Paper, Fig. 1), and therefore choose structured channel pruning. They use a GA because it can explore many candidates and balance several goals at once (Paper). A usual GA string over millions of weights would be too long, so the chromosome holds one pruning ratio per layer instead.

For retraining, the authors argue that FT and LRW ignore the original model’s outputs and treat all wrong classes alike. Their method, RKD, uses the original model as teacher and the pruned model as student (Paper). The paper does not discuss manual per-layer tuning in detail; its stated open challenge is finding a good balance between model size and accuracy. The main contributions are:

- A compact chromosome of layer-wise pruning ratios, with a starting ratio that grows with the number of channels plus noise in [−0.2, 0.2] (Paper).
- Two crossover operators (uniform and arithmetic) applied to two halves of the population, with mutation and top-half selection (Paper).

- A fitness function that combines accuracy, parameter count and FLOPs, with $\alpha = 1$, $\beta = 4$ and $\gamma = 4$ (Paper).
- RKD: a weighted sum of a label loss and a KL-divergence loss, with a temperature falling linearly from 20 to 1 (Paper).
- Evaluation on a video gesture model and on ResNet-32 and ResNet-56 (Paper).

Table 2.1: Comparison of the four approaches (the Basis column separates the paper's statements from general comparisons)

Approach	Core idea	Strengths	Limitations	Basis
Conventional pruning	Remove weights (unstructured) or whole filters/channels (structured) with low importance [3], [4], [5]	Fewer parameters; structured form keeps dense tensors and suits standard hardware	Per-layer ratios must be chosen; accuracy drops; unstructured form gave almost no CPU speed-up [1]	Paper [1] Sec. 1–2; general
GA-based pruning	Encode per-layer ratios as a chromosome and evolve them by crossover, mutation and selection	Searches many configurations without gradients; can weigh accuracy, size and FLOPs together	Needs many fitness evaluations; extra design choices (population, rates, weights)	Paper Sec. 2.3, 3.1; cost point is *analysis*
Knowledge Distillation	A student learns from the teacher's softened outputs as well as from labels [2]	Passes on class-similarity information; can speed up recovery	Needs a teacher; adds temperature and loss-weight settings	Paper Sec. 2.4, 3.2; [2]
Combined framework	GA finds P_{best}; channels are pruned; the student is retrained with KD	Reported >90% fewer parameters and FLOPs on the gesture model; RKD matched LRW-level accuracy in 150 vs 200 epochs	Only three networks tested; no ablation separating GA from KD (*analysis*)	Paper Sec. 3–4; limits are *analysis*

2.3 Theoretical Background

2.3.1 Convolutional Neural Networks

A CNN is built from a few repeated parts. A **convolutional layer** slides filters over its input and produces feature maps; each filter creates one output channel. An **activation function**, commonly ReLU, $f(x) = \max(0, x)$, adds non-linearity. **Pooling** shrinks the feature maps. **Fully connected layers** with a softmax turn the features into class probabilities. In this report a *channel* is one feature map and a *filter* is the set of weights that produces it, so removing a filter removes its output channel. For a convolutional layer with C_{in} input channels, C_{out} filters and a $K_h \times K_w$ kernel, the number of weights (bias ignored) and the multiply–accumulate operations (MACs) for an $H_{out} \times W_{out}$ output map are:

$$P_{conv} = C_{out} \cdot C_{in} \cdot K_h \cdot K_w \tag{1}$$

$$MACs_{conv} = H_{out} \cdot W_{out} \cdot C_{out} \cdot C_{in} \cdot K_h \cdot K_w \tag{2}$$

Example (general): a layer with $C_{in} = 64$, $C_{out} = 128$, a 3×3 kernel and a 32×32 output map has $128 \times 64 \times 9 = 73{,}728$ weights and $32 \times 32 \times 128 \times 64 \times 9 = 75{,}497{,}472$ MACs. If a fraction p_l of the output channels of layer l and a fraction p_{l-1} of those of the layer before it are removed, the layer keeps $(1 - p_l)(1 - p_{l-1})$ of its weights (*analysis*):

$$P' = (1 - p_l)(1 - p_{l-1})P_{conv} \tag{3}$$

Pruning 70% of the channels in every layer therefore leaves 0.3 × 0.3 = 9% of the weights, a 91% reduction. This only illustrates why a roughly 90% parameter cut needs about 70% of channels removed; the paper does not report its per-layer ratios.

2.3.2 Structured Channel Pruning

Structured channel pruning removes entire channels together with their filters, whereas unstructured pruning zeroes single weights (Figure 2.1 and Table 2.2). The benefit is real speed-up without special hardware. The trade-off is that removing a whole channel is a bigger change than zeroing one weight, so accuracy can fall more and retraining is needed. The paper scores channel importance with a Taylor expansion [5]; the formula is not reproduced in [1], but the idea is to estimate how much the loss would change if the channel were removed.

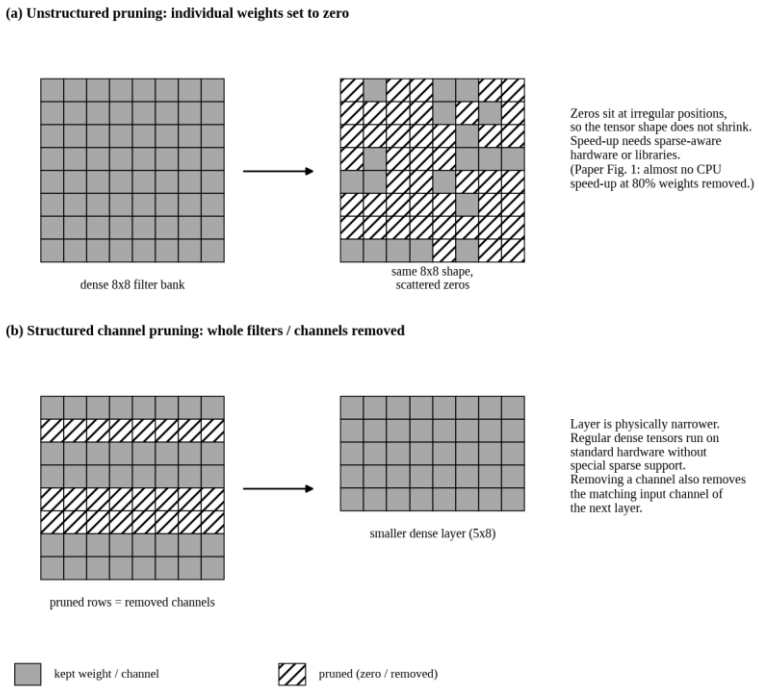


Figure 2.1: Structured versus unstructured pruning (conceptual illustration). *Source: original diagram by the author; the HoloLens observation is from Fig. 1 of [1].*

Table 2.2: Unstructured versus structured pruning

Aspect	Unstructured (weight) pruning	Structured (channel) pruning
What is removed	Individual weights, at arbitrary positions	Whole filters / channels (and the matching input channels of the next layer)
Tensor shape	Unchanged; the tensor becomes sparse	Smaller dense tensors
Hardware need	Sparse-aware support to gain speed	Runs on standard hardware
Latency effect	Often small (paper Fig. 1: almost no CPU speed-up at 80% removal) (Paper)	Real reduction (paper Table 1: 40.63–54.17%) (Paper)
Accuracy risk	Finer granularity, usually gentler (*general*)	Coarser, so a higher risk of accuracy loss (*general*)

2.3.3 Genetic Algorithm

A Genetic Algorithm imitates natural selection. Its parts, as used for pruning, are:

- **Population and chromosome:** a chromosome is one candidate solution, here a vector with one pruning ratio (gene) per layer. The population is the set of M such vectors.
- **Initial population:** M different vectors, built from a size-aware starting ratio plus random noise.
- **Fitness evaluation:** a score for each vector; a higher score means a better balance of accuracy, size and computation.
- **Selection:** the better individuals are kept for the next generation (the paper keeps the top half).
- **Crossover:** two parents are mixed into an offspring that inherits genes from both.
- **Mutation:** a small random change to some genes, which keeps diversity and avoids local optima.
- **New generation and stopping:** the steps repeat until a predefined number of generations (10 to 20 in the paper).

Example (Paper, Example 1): a model with three layers has ChannelCounts = [128, 64, 256]. The chromosome [0.2, 0.32, 0.55] means 20%, 32% and 55% of the channels are pruned in layers 1, 2 and 3, which is 25.6, 20.48 and 140.8 channels, so a rounding rule is needed; the paper does not state one. Table 2.3 shows how the operators act on such vectors, with illustrative numbers chosen by the author.

Table 2.3: Illustration of GA operators on a three-layer chromosome (hypothetical numbers, not from the paper)

Operation	Setting	Result
Parents	P_i = [0.20, 0.32, 0.55]; P_j = [0.40, 0.10, 0.35]	–
Uniform crossover	selection_rate = 0.5; $rand_C$ = (0.3, 0.8, 0.1)	[0.20, 0.10, 0.55]
Arithmetic crossover	r = 0.7	[0.26, 0.254, 0.49]
Mutation	gene 2 mutates; mutation_factor = +0.05	[0.26, 0.304, 0.49]

2.3.4 Knowledge Distillation

A normal classifier trained on hard labels learns only “this is class 3”. A teacher’s output probabilities also say “and it looks a little like class 7”. These **soft targets** carry extra information, and the softmax with **temperature** T makes them softer [2]:

$$q_k = \frac{exp(z_k / T)}{\sum_j exp(z_j / T)} \tag{4}$$

Here z_k is the logit (raw output) for class k. A higher T gives a flatter distribution. *Example (general):* for logits z = [3, 1, 0.2], T = 1 gives probabilities [0.836, 0.113, 0.051], while T = 4 gives [0.475, 0.288, 0.236]. Training combines two standard losses, cross-entropy against the true labels y and KL divergence between the softened teacher and student outputs [10]:

$$CE = -\sum_k y_k \, log \, \hat{y}_k \tag{5}$$

$$KL(P \parallel Q) = \sum_k P_k \, log \left(\frac{P_k}{Q_k}\right) \tag{6}$$

Equations (4)–(6) are standard background. The equations that the paper itself reports are given in Chapter 4 and are marked as such.

2.4 Synthesis and Research Gap

The literature offers strong pruning criteria, retraining methods and distillation. What remains open is how to combine them so that layer-wise ratios are found automatically and accuracy is recovered quickly. The reference paper answers this with a GA plus RKD, but it leaves the separate contribution of each part untested and omits several settings. This case study therefore (i) restates the method and checks its numbers against the paper, (ii) starts a reproducible CIFAR-10 experiment with a measured baseline, and (iii) lays out the comparison needed to isolate the effect of the GA and of distillation.

CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

3. System Requirements and Specifications

3.1 Hardware and Software Requirements

3.1.1 Environment of the Reference Paper

The authors used Python with the PyTorch library on Windows 10 with an Intel i5-13600 CPU, an NVIDIA RTX 4090 GPU and 64 GB of RAM (Paper). Inference timings in Fig. 1 of [1] were taken on a HoloLens CPU; the device for the timings in Table 1 is not specified in the reference paper.

3.1.2 Environment of this Study

The student's experiment runs in a Google Colab notebook on a GPU runtime. The versions below are those printed by the notebook (Notebook). The package upgrade cell also printed dependency warnings: pandas 3.1.0 was installed although google-colab requires pandas 2.2.3 and the cudf packages require pandas below 2.4. These did not stop the experiment, but they should be fixed or pinned for a clean reproduction.

Table 3.1: Software and hardware environment of the notebook (Notebook)

Component	Version / detail	Role
Runtime	Google Colab, Python 3.13	Execution environment
GPU	Tesla T4, 15.64 GB memory	Training
PyTorch	2.14.1 (CUDA 13.0 build); CUDA available: True	Model and training
torchvision	0.29.1	CIFAR-10 loading and transforms
scikit-learn	1.9.1	Macro-F1 (planned)
matplotlib, pandas	3.11.2, 3.1.0	Plots and tables

3.2 Dataset Description and Characteristics

The notebook uses **CIFAR-10** [14]: 60,000 colour images of 32 × 32 pixels in ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The 50,000 training images are split with a fixed random seed (42) into 45,000 for training and 5,000 for validation, and the 10,000 official test images are kept for final evaluation (Notebook). Images are converted to tensors and normalised with the per-channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616); no data augmentation is applied. Batch size is 128 with two loader workers.

Table 3.2: Data used in the reference paper and in this study

Item	Reference paper [1] (Paper)	This study (Notebook)	Notes
Gesture data	EgoGesture, 83 hand gestures [13]	Not used	Video data
Image data	CIFAR-10 and CIFAR-100 [14]	CIFAR-10 only	10 classes, 32 × 32
Models	R(2+1)D-18 (classifiers A–D); ResNet-32, ResNet-56	Small 3-conv CNN (Chapter 5)	Baseline / teacher

Item	Reference paper [1] (Paper)	This study (Notebook)	Notes
Splits	Not specified in the reference paper	45,000 / 5,000 / 10,000	Seed 42 for the split

3.3 Evaluation Metrics

- **Accuracy and macro-F1:** top-1 accuracy; macro-F1 is the unweighted mean of the per-class F1 scores (planned for the CIFAR-10 test set).
- **Parameters (|Param|) and model size:** the count of learnable parameters and the size of the saved weights.
- **Computation (|FLOP|):** floating-point operations; this report counts multiply–accumulate operations (MACs) and states the convention, since [1] does not state its own.
- **Inference time:** latency in milliseconds, measured on the same device for every model.

Reductions are reported as percentage reductions, and accuracy changes as percentage-point differences; the difference between the two is explained with a worked example in Section 6.4.1.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

4. Methodology and System Design

4.1 Methodology Workflow

The framework of [1] runs in ten stages, summarised in Table 4.1 and drawn in Figure 4.1. Stages 2 to 6 are the GA search, stages 7 and 8 compress and recover the model, and stages 9 and 10 evaluate it.

Table 4.1: The ten stages of the hybrid methodology (Paper, Sec. 3)

#	Stage	Purpose	Expected output
1	Select original CNN	Provide the trained baseline whose accuracy must be kept; it also serves as the teacher	Baseline weights and accuracy
2	Encode pruning ratios	Represent one pruning ratio per layer as a chromosome	Vector of length N
3	Initialise population	Create M diverse candidates: size-aware starting ratio plus noise	Init = $[P_1, \ldots, P_M]$
4	Evaluate fitness	Score each candidate on accuracy, |Param| and |FLOP|	Fitness per candidate
5	Apply GA operators	Crossover (two groups), mutation and top-half selection, repeated for the set number of generations	Evolved population
6	Select P_{best}	Take the best individual found	Best pruning vector
7	Prune channels	Remove channels in each layer as P_{best} says, least important first (Taylor score)	Pruned (student) network
8	Retrain with KD	Recover accuracy using the label loss plus the teacher's soft targets	Retrained student
9	Evaluate	Measure accuracy, |Param|, |FLOP| and inference time	Metric values
10	Compare	Compare with the baseline and with other methods	Reduction and accuracy change

How the accuracy term is obtained for each GA candidate (for example, whether a candidate is briefly retrained before scoring) is **not specified in the reference paper**. This matters for cost and is discussed in Section 6.5.

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

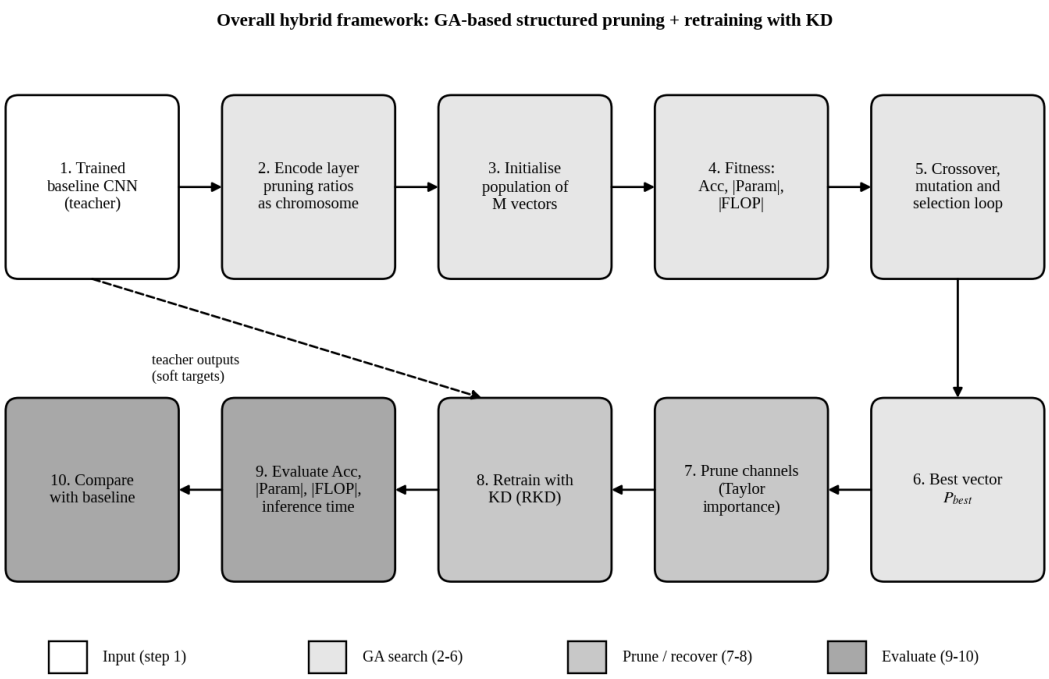


Figure 4.1: Overall hybrid framework, from the baseline CNN to the compared compressed model.
Source: original diagram by the author, based on Sec. 3 and Fig. 4 of [1].

4.2 Proposed Hybrid Pipeline: Mathematical Formulation

Every equation in Sections 4.2.1 to 4.2.5 is taken from the reference paper [1]; the report's equation number is followed by the paper's EQ number. Table 4.2 defines the symbols.

Table 4.2: Symbols and their values (Paper, Sec. 3–4)

Symbol	Meaning	Value in paper
N, $channel_j$	Number of layers; number of channels in layer j	Model-dependent
P_i, $pruned_{i,j}$	Pruning vector of individual i; pruning ratio of layer j	Evolved by the GA
$NF_{i,j}$	Noise factor, uniform random	Range [−0.2, 0.2]
κ, λ	User parameters of the initial pruning ratio	κ = 150, λ = 0.2
M	Population size	Not specified in the reference paper
Generations	Number of GA generations	10 to 20
selection_rate, $rand_C$	Threshold and random draw for uniform crossover	Rate: not specified
r	Random weight of arithmetic crossover	Distribution not specified
mutation_rate, mutation_factor, $rand_M$	Probability, size of change and random draw for mutation	Rate and factor: not specified
Acc, |Param|, |FLOP|	Accuracy; parameter count; floating-point operations	Measured per model
α, β, γ	Fitness weights for accuracy, size and operations	α = 1, β = 4, γ = 4
z_S, z_T, T	Student logits, teacher logits, temperature	T starts at T_0 = 20
δ, E	Weight of the label loss; total retraining epochs	δ = 0.5, E = 150

4.2.1 Chromosome and Initialisation

The channel counts of a model with N layers and the pruning vector of the i-th individual are (paper EQ 1 and EQ 2):

$$ChannelCounts = [channel_1, channel_2, \dots, channel_N] \quad (7)$$

$$P_i = [pruned_{i,1}, pruned_{i,2}, \dots, pruned_{i,j}, \dots, pruned_{i,N}] \quad (8)$$

Layers with more channels can tolerate more pruning, so the starting ratio grows with the channel count (paper EQ 4). Noise gives each individual a different value (paper EQ 3 and EQ 5), and the population collects M such vectors (paper EQ 6):

$$Initial_Pruning_Ratio_j = \left(1 - e^{-\lambda \times \frac{channel_j}{\kappa}}\right) \quad (9)$$

$$pruned_{i,j} = \left(1 - e^{-\lambda \times \frac{channel_j}{\kappa}}\right) + NF_{i,j} \quad (10)$$

$$Init = [P_1, P_2, \dots, P_M] \quad (11)$$

With $\kappa = 150$ and $\lambda = 0.2$, Equation (9) gives starting ratios of 0.082, 0.157, 0.289 and 0.495 for layers of 64, 128, 256 and 512 channels (computed from the paper's equation; Figure 4.4a). Equation (10) then adds noise between −0.2 and +0.2. How a ratio that falls outside [0, 1] after noise is handled is not specified.

4.2.2 Crossover and Mutation

The population is split into two groups: one uses uniform crossover [8] and the other arithmetic crossover [9]. In uniform crossover each gene comes from parent P_i if a random number $rand_C$ is below selection_rate, and otherwise from parent P_j (paper EQ 7). In arithmetic crossover a random weight r blends the two parents (paper EQ 8):

$$pruned_{i,j}^{Offspring} = pruned_{i,j}^{Pi}, \ if\ rand_C < selection_rate \quad (12)$$

$$pruned_{i,j}^{Offspring} = pruned_{i,j}^{Pj}, \ otherwise$$

$$pruned_{i,j}^{Offspring} = r \cdot pruned_{i,j}^{Pi} + (1 - r) \cdot pruned_{i,j}^{Pj} \quad (13)$$

For mutation, each gene draws $rand_M$. If it is below mutation_rate, the gene changes by a constant mutation_factor (paper EQ 9):

$$pruned'_{i,j} = pruned_{i,j} + mutation_factor, \ if\ rand_M < mutation_rate \quad (14)$$

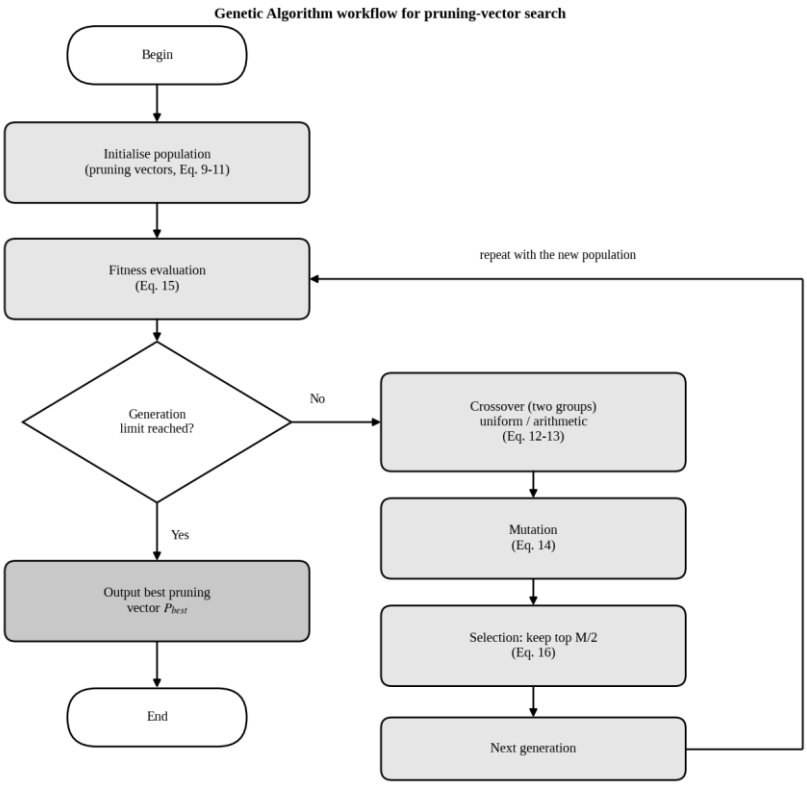


Figure 4.2: Genetic Algorithm workflow for searching the pruning vector. *Source: original diagram by the author, based on Sec. 3.1 and Fig. 2 of [1].*

4.2.3 Fitness Function and Selection

The fitness value balances accuracy, model size and computation (paper EQ 10):

$$Fitness = \alpha \cdot Acc + \frac{\beta}{log(|Param|)} + \frac{\gamma}{log(|FLOP|)} \quad (15)$$

Here Acc is the model's accuracy, |Param| its parameter count and |FLOP| its floating-point operations, with α = 1, β = 4 and γ = 4 (Paper). The printed equation in [1] writes the third coefficient as "r", while the text defines it as γ; this report reads it as γ (a typographical inconsistency in the source). Because the size and operation terms are inverse logarithms, fewer parameters or FLOPs give a higher fitness, but the reward grows slowly (*analysis*). The log base, the scale of Acc (fraction or percent) and how Acc is measured are not specified in the reference paper. Selection ranks all individuals by fitness and keeps the top half (paper EQ 11):

$$I_{current} = [P_1, \dots, P_{M/2}] \quad (16)$$

After the final generation, the best individual is called P_{best} and is used to prune the model by channel importance (Paper).

4.2.4 Structured Channel Pruning

Given P_{best}, each layer j loses the fraction $pruned_{best,j}$ of its channels. The channels removed are the least important ones according to a Taylor-expansion score [5] (Paper). Removing an output channel also removes the matching input weights of the next layer, which is why parameters and FLOPs fall faster than the channel count (Equation 3).

4.2.5 Retraining with Knowledge Distillation (RKD)

The student is the pruned model and the teacher is the original model (Figure 4.3). The loss has a label part (paper EQ 12) and a distillation part (paper EQ 13), combined with a weight δ (paper EQ 14):

$$Loss1 = Loss(Softmax(Output_Student), Target\ Label) \quad (17)$$

$$Loss2 = T^2 \times KL_div\left(Softmax\left(\frac{Output_Student}{T}\right), Softmax\left(\frac{Output_Teacher}{T}\right)\right) \quad (18)$$

$$Loss_Total = \delta * Loss1 + (1 - \delta) * Loss2 \quad (19)$$

The paper describes Loss1 as a traditional loss "like cross-entropy", so Equation (5) is its usual form. Loss2 lists the student distribution first and the teacher second; the paper does not spell out the argument convention of KL_div, and common software libraries order these arguments differently. The factor T^2 is the usual scaling that keeps the gradient size comparable when T changes [2] (*general background*). The temperature falls linearly from T_0 to 1 over the E retraining epochs (paper EQ 15), with $T_0 = 20$, E = 150 and $\delta = 0.5$ (Paper); Figure 4.4b plots this schedule:

$$T(epoch) = T_0 - \frac{T_0 - 1}{E} \cdot epoch \quad (20)$$

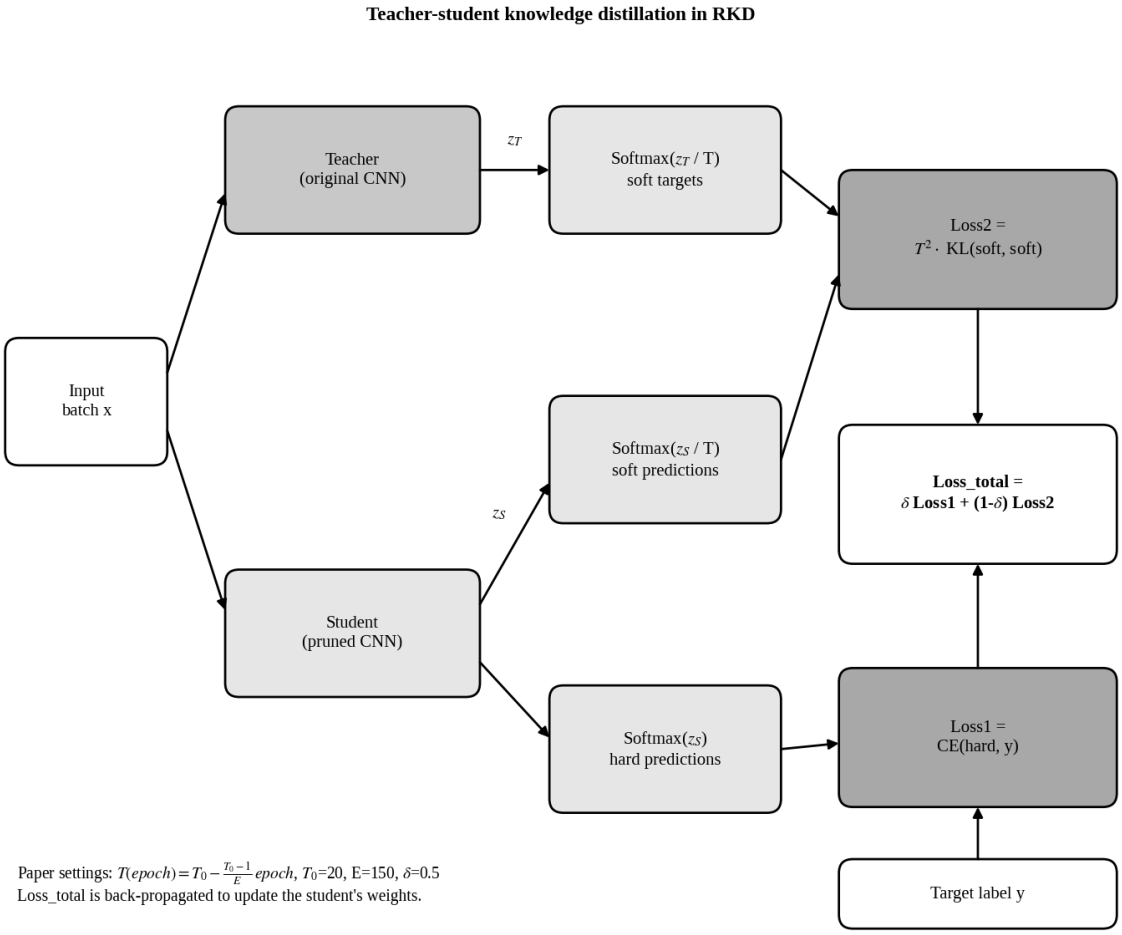


Figure 4.3: Teacher–student knowledge distillation used for retraining (RKD). *Source: original diagram by the author, based on Sec. 3.2 and Fig. 3 of [1].*

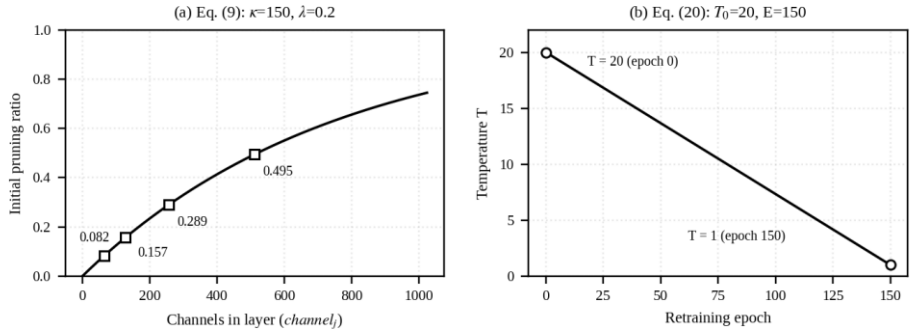


Figure 4.4: Computed curves of (a) the initial pruning ratio and (b) the temperature schedule. *Source: computed by the author from Equations (9) and (20) using κ, λ, T_0 and E reported in [1].*

4.3 Algorithm

Algorithm 4.1 gives the complete hybrid procedure, written by the author from the description in [1].

Line	**Algorithm 4.1: GA-based structured pruning with retraining by KD (after [1])**
1	**Input:** trained CNN F (teacher), training data, M, G, κ, λ, α, β, γ, selection_rate, mutation_rate, mutation_factor, δ, T_0, E
2	**Output:** compressed CNN F_s
3	Read ChannelCounts = [$channel_1$, …, $channel_N$] from F
4	For each individual i = 1..M and layer j = 1..N:
5	$init_j \leftarrow 1 - \exp(-\lambda \cdot channel_j / \kappa)$ (Eq. 9)
6	$pruned[i, j] \leftarrow init_j + U(-0.2, 0.2)$ (Eq. 10)
7	Population ← [P_1, …, P_M]; evaluate the fitness of each individual (Eq. 15)
8	For generation g = 1..G (G = 10 to 20 in the paper):
9	Split the population into two groups
10	Group 1: uniform crossover; group 2: arithmetic crossover (Eq. 12, 13)
11	For each gene: if $rand_M$ < mutation_rate, add mutation_factor (Eq. 14)
12	Evaluate the fitness of all individuals
13	Rank by fitness and keep the top M/2 (Eq. 16)
14	Form the next generation (refilling to size M is not specified in [1])
15	P_{best} ← best individual found
16	F_s ← remove channels from F by the ratios in P_{best}, least important first
17	For epoch = 0..E (E = 150):
18	$T \leftarrow T_0 - (T_0 - 1)/E \cdot epoch$ (Eq. 20)
19	For each mini-batch (x, y): $z_T \leftarrow F(x)$; $z_S \leftarrow F_s(x)$
20	$L1 \leftarrow CE(softmax(z_S), y)$ (Eq. 17)
21	$L2 \leftarrow T^2 \cdot KL(softmax(z_S/T), softmax(z_T/T))$ (Eq. 18)
22	$L \leftarrow \delta \cdot L1 + (1 - \delta) \cdot L2$; update the weights of F_s (Eq. 19)
23	Return F_s

Selection needs fitness values and a full-size population is needed in the next round; [1] does not detail either step, so lines 13 and 14 reflect the author's reading.

5. Implementation Details and Experimental Setup

5.1 Implementation Environment

All code runs in the Colab notebook described in Section 3.1.2 using PyTorch [17] on a Tesla T4 GPU. The notebook so far covers environment checks, data loading, the baseline CNN, its training, and the start of a channel-importance step. Section 8.1 lists the notebook cells and their status.

5.2 Baseline CNN (Teacher) Architecture

The baseline CNN printed by the notebook has three 3 × 3 convolutional layers (32, 64 and 128 channels), each followed by batch normalisation, a shared 2 × 2 max-pooling module, and two fully connected layers (256 and 10 units). It has **620,810** parameters (Notebook). The notebook output does not show the forward function. The fc1 input size of 2048 equals 128 × 4 × 4, which means three pooling steps on a 32 × 32 input, so this report infers that pooling follows each convolution (*analysis*). Table 5.1 gives the parameters from the printed layers and the MACs computed from Equations (1) and (2) under that inference.

Table 5.1: Layer-wise parameters and MACs of the baseline CNN (parameters: Notebook; MACs: derived)

Layer	Output (before pooling)	Parameters	MACs	Share of MACs
conv1 (3→32)	32 × 32 × 32	896	884,736	8.16%
bn1 (32)	–	64	–	–
conv2 (32→64)	64 × 16 × 16	18,496	4,718,592	43.49%
bn2 (64)	–	128	–	–
conv3 (64→128)	128 × 8 × 8	73,856	4,718,592	43.49%
bn3 (128)	–	256	–	–
fc1 (2048→256)	256	524,544	524,288	4.83%
fc2 (256→10)	10	2,570	2,560	0.02%
Total	–	**620,810**	**10,848,768**	**100%**

Parameter counts add up to the 620,810 printed by the notebook. MACs ignore biases and batch-norm operations.

The two large convolutions, conv2 and conv3, hold 86.99% of the computation but only 14.9% of the parameters. The first fully connected layer holds 84.49% of the parameters (Figure 5.1). Pruning conv3 channels shrinks fc1 as well, because each conv3 channel feeds 4 × 4 = 16 fc1 inputs, which is 4,096 weights (*analysis*).

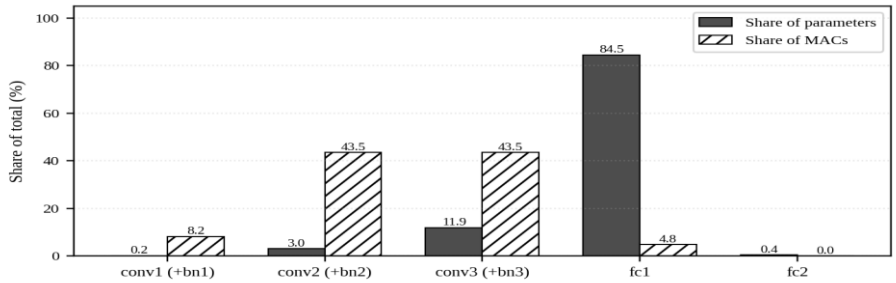


Figure 5.1: Share of parameters and MACs by layer for the baseline CNN. *Source: derived from Table 5.1.*

5.3 Training Protocol

Table 5.2: Training configuration of the baseline CNN (Notebook)

Setting	Value
Loss function	Cross-entropy (nn.CrossEntropyLoss)
Optimiser	AdamW, learning rate 0.001, weight decay 1e-4
Learning-rate schedule	Cosine annealing, T_{max} = 8 epochs, stepped once per epoch
Epochs and batch size	8 epochs; batch size 128
Precision	Mixed precision (float16 autocast with GradScaler)
Model selection	Best validation accuracy checkpoint saved as best_teacher.pth
Data split	45,000 train / 5,000 validation; split seed 42; test set untouched
Augmentation	None (tensor conversion and normalisation only)

5.4 Channel-Importance Step

The last notebook cell reloads the best checkpoint and registers forward hooks on conv1, conv2 and conv3 that store each layer's activations and keep their gradients with retain_grad. This is the first step of a Taylor-style importance score of the kind used in [1] and [5]. The supplied log ends inside this cell, so no importance scores are available yet.

5.5 Planned Experimental Design

The baseline above is model M1 of a five-way comparison. The remaining configurations, listed in Table 6.5 and drawn in Figure 5.3, are **Proposed** and have not been run. Figure 5.2 shows the planned software structure. Table 5.3 compares the paper's settings with the plan.

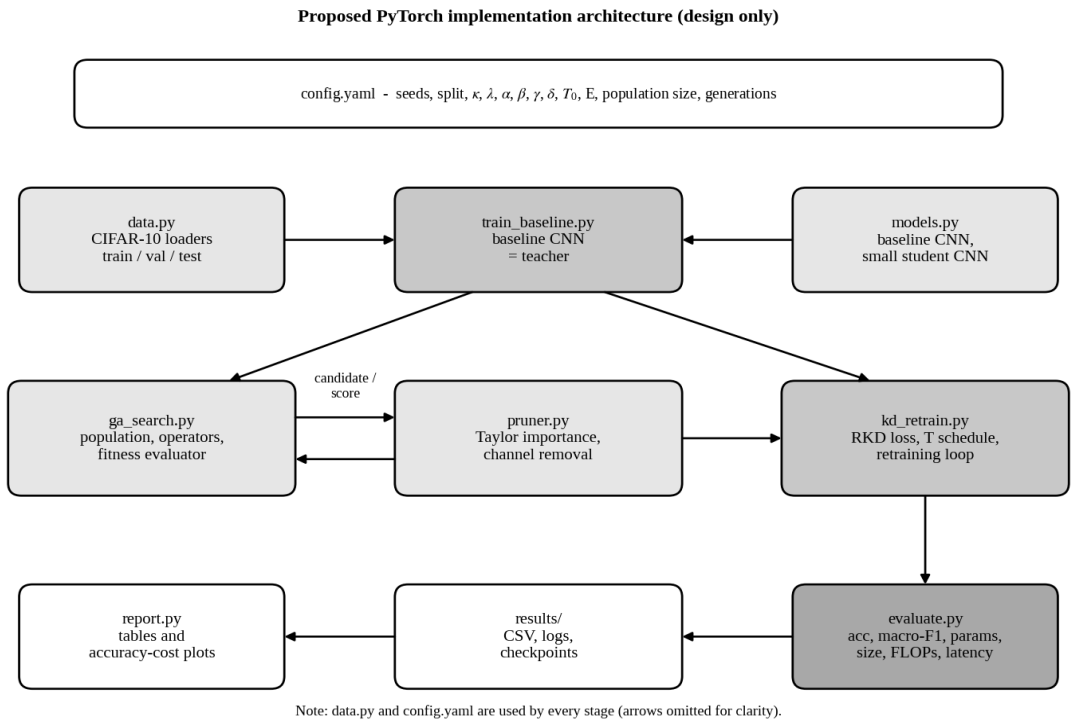


Figure 5.2: Proposed PyTorch implementation architecture (design only). *Source: original design by the author (Proposed; not implemented).*

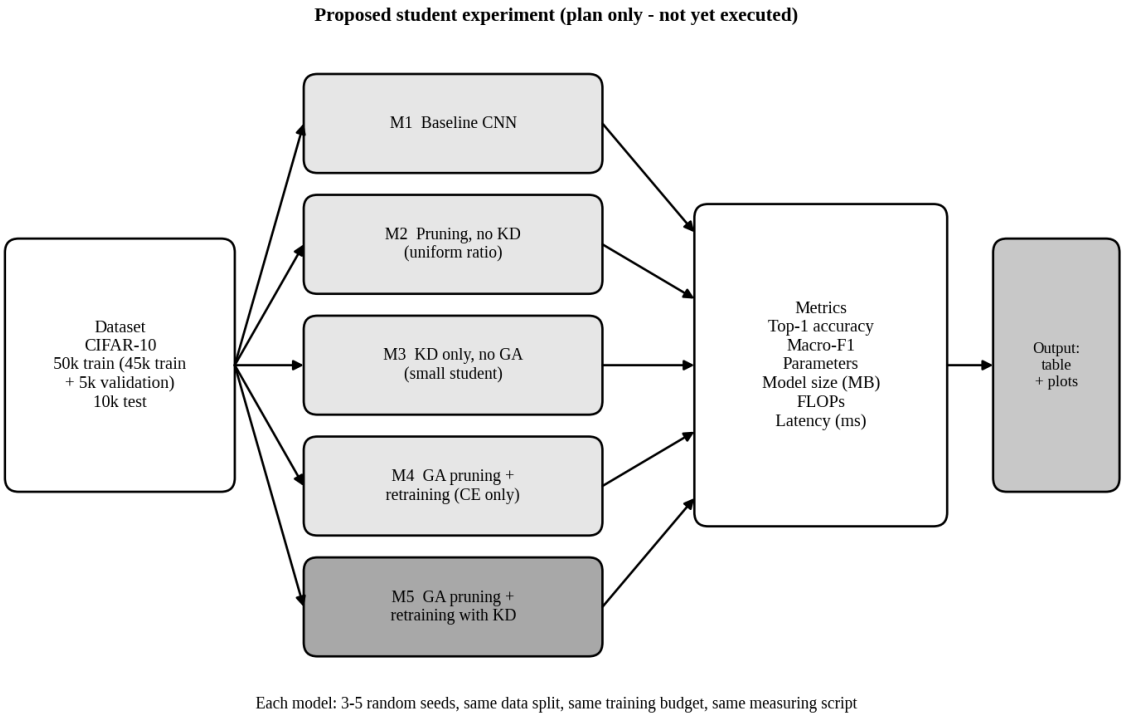


Figure 5.3: Proposed experimental evaluation (plan only; not yet executed). *Source: original design by the author (Proposed).*

Table 5.3: Settings of the reference paper compared with the plan for this study

Setting	Reference paper [1] (Paper)	This study
κ, λ	150, 0.2	Same (Proposed)
α, β, γ	1, 4, 4	Same (Proposed)
δ, T_0	0.5, 20	Same (Proposed)
Retraining epochs E	150	Chosen within Colab limits and kept equal for all retrained models (Proposed)
Population size	Not specified in the reference paper	20 (Proposed, author's choice)
Generations	10 to 20	10 (Proposed)
Fitness accuracy	Not specified in the reference paper	Validation split after a short fine-tune (Proposed)
Seeds	Not specified in the reference paper	3 to 5 seeds, mean ± std (Proposed)

6. Results and Performance Analysis

6.1 Overview

This chapter presents three kinds of results, kept apart: the results reported in the reference paper (Paper), the numbers measured in the student's notebook (Notebook), and values derived from them (*analysis*). The paper's compression results come from different models and datasets than the notebook, so they are not directly comparable; they define the target that the planned CIFAR-10 experiment aims to approach.

6.2 Results Reported in the Reference Paper

6.2.1 R(2+1)D-18 on EgoGesture

Table 6.1 reproduces Table 1 of [1] for the four classifiers of the self-distilled R(2+1)D-18 model. "Acc drop" is the difference in accuracy in percentage points.

Table 6.1: Original versus pruned R(2+1)D-18 classifiers on EgoGesture (Paper, Table 1)

Clf.	Acc (%) orig → pruned	Acc drop (pp)	|Param| (M) orig → pruned	|Param| ↓ (%)	|FLOP| (G) orig → pruned	|FLOP| ↓ (%)	Time (ms) orig → pruned	Time ↓ (%)
A	85.09 → 77.12	7.97	1.17 → 0.11	90.22	29.28 → 2.66	90.92	2.4 → 1.1	54.17
B	91.34 → 89.01	2.33	2.71 → 0.25	90.71	34.32 → 3.11	90.94	2.8 → 1.4	50.00
C	92.12 → 90.14	1.98	8.73 → 0.79	90.98	38.35 → 3.47	90.96	3.2 → 1.9	40.63
D	**92.24 → 90.42**	**1.82**	**31.34 → 2.81**	**91.02**	**40.52 → 3.66**	**90.96**	**3.6 → 2.1**	**41.67**

Source: [1], Table 1. pp = percentage points; ↓ = reduction relative to the original.

For classifier D, the paper reports accuracy of 92.24% before and 90.42% after pruning, a 1.82 percentage-point drop. Parameters fall from 31.34 M to 2.81 M (91.02% reduction) and FLOPs from 40.52 G to 3.66 G (90.96% reduction). Inference time falls from 3.6 ms to 2.1 ms. All values match Table 1 of [1] and its abstract. Every classifier loses more than 90% of both parameters and FLOPs, and inference time falls by 40.63% to 54.17% (Paper).

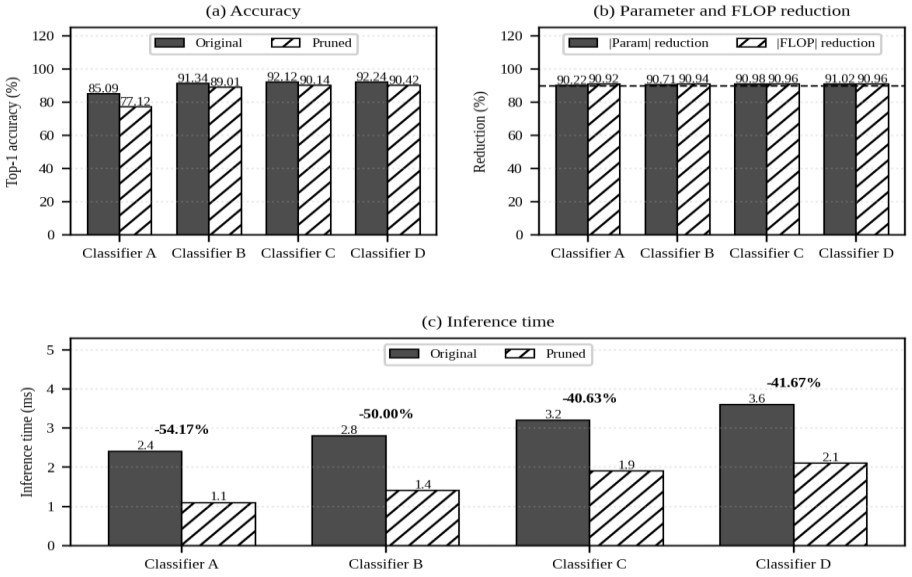


Figure 6.1: Reported results for classifiers A to D: (a) accuracy, (b) parameter and FLOP reduction, (c) inference time. *Source: values from Table 1 of [1].*

6.2.2 ResNet-32 and ResNet-56 on CIFAR-10 and CIFAR-100

Table 6.2: Pruned ResNets compared with two earlier methods (Paper, Table 2)

Dataset	Model	Approach	Ori. Acc (%)	New Acc (%)	Acc drop (pp)	|Param| ↓ (%)	|FLOP| ↓ (%)
CIFAR-10	ResNet-32	Cheng 2023 [16]	92.63	91.02	1.61	–	41.50
		Cheng 2021 [15]	92.63	92.14	0.49	–	53.20
		Ours [1]	92.89	92.17	0.72	50.43	58.14
	ResNet-56	Cheng 2023 [16]	93.59	91.67	1.92	–	54.60
		Ours [1]	93.22	92.83	0.39	63.53	63.85
CIFAR-100	ResNet-32	Cheng 2021 [15]	70.28	69.44	0.84	–	41.50
		Ours [1]	69.94	68.88	1.06	43.06	49.37
	ResNet-56	Cheng 2023 [16]	71.07	70.73	0.34	–	52.60
		Ours [1]	71.04	70.69	0.35	59.93	62.23

Source: [1], Table 2. "–" means not reported. Rows labelled [15] and [16] are cited as [4] and [5] in the paper.

On CIFAR-10, the pruned ResNets keep accuracy within 0.72 and 0.39 points and reduce FLOPs by 58.14% and 63.85%, more than the two earlier methods in the same table. On CIFAR-100, ResNet-32 ends slightly below [15] in accuracy (68.88% against 69.44%) but with a larger FLOP reduction (49.37% against 41.50%). For ResNet-56, the drop is 0.35 points with a 62.23% FLOP reduction, comparable to [16] (Paper). Table 6.3 compares RKD with LRW [6].

Table 6.3: Retraining with RKD versus LRW [6] (Paper, Table 3)

Dataset	Model	Ori. Acc (%)	Approach	New Acc (%)	Acc drop (pp)	Epochs	|Param| / |FLOP| ↓ (%)
CIFAR-10	ResNet-32	92.89	LRW [6] Ours	92.02 92.17	0.87 0.72	200 150	50.43 / 58.14
CIFAR-10	ResNet-56	93.22	LRW [6] Ours	92.88 92.83	0.34 0.39	200 150	63.53 / 63.85
CIFAR-100	ResNet-32	69.94	LRW [6] Ours	68.77 68.88	1.17 1.06	200 150	43.06 / 49.37
CIFAR-100	ResNet-56	71.04	LRW [6] Ours	70.60 70.69	0.44 0.35	200 150	59.93 / 62.23

Source: [1], Table 3.

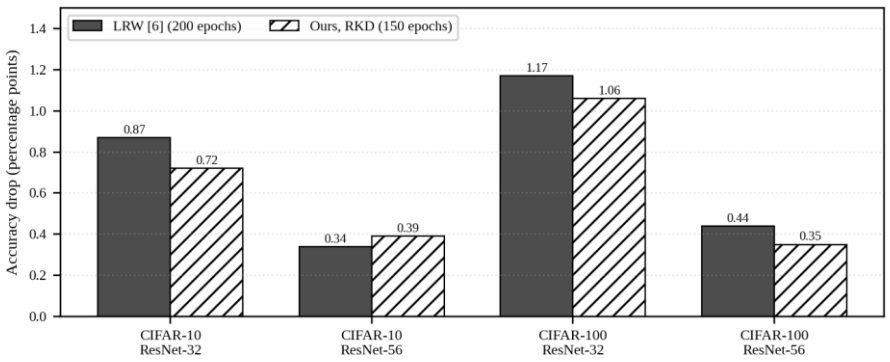


Figure 6.2: Accuracy drop of LRW (200 epochs) and the proposed RKD (150 epochs). *Source: values from Table 3 of [1].*

RKD reaches an accuracy drop similar to LRW in 150 epochs instead of 200, which is 25% fewer epochs. It is slightly better in three of the four settings and slightly worse for ResNet-56 on CIFAR-10 (0.39 against 0.34 points) (Paper).

6.3 Results from the Baseline Experiment of this Study

Table 6.4 lists the per-epoch log printed by the notebook when the baseline CNN was trained for eight epochs on a Tesla T4 GPU (Notebook).

Table 6.4: Training log of the baseline CNN on CIFAR-10 (Notebook)

Epoch	Training loss	Training accuracy (%)	Validation accuracy (%)	Time (s)
1	1.2486	54.95	64.76	13.5
2	0.8455	70.18	68.58	11.6
3	0.6872	75.88	73.98	11.6
4	0.5812	79.68	75.88	12.1
5	0.4793	83.54	76.12	11.5
6	0.3913	86.84	78.04	11.8
7	0.3242	89.44	78.62	11.5
8	**0.2839**	**91.19**	**79.44**	**11.6**

Source: output of the training cell of the student's notebook. Best validation accuracy: 79.44% (epoch 8).

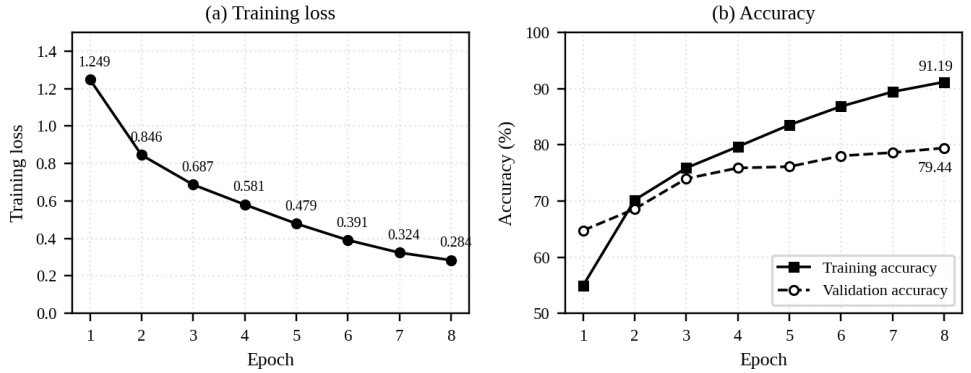


Figure 6.3: Training loss and accuracy of the baseline CNN over eight epochs. *Source: values from Table 6.4 (Notebook).*

Training loss fell steadily from 1.2486 to 0.2839, and validation accuracy rose from 64.76% to 79.44%, which is the best value and the one saved as the teacher checkpoint. Epochs took about 11.9 s on average (95.2 s in total, *analysis* from the log). Table 6.5 collects the baseline metrics and shows the status of the planned comparison.

Table 6.5: Status of the planned five-model comparison (M1 from the notebook; M2 to M5 Proposed)

ID	Configuration	Val acc (%)	Test acc (%)	Macro-F1	|Param|	Size (MB)	MACs	Latency (ms)
M1	Original CNN baseline (teacher)	79.44	To be measured	To be measured	620,810	≈2.48 (derived)	≈10.85 M (derived)	To be measured
M2	Pruning without KD (uniform ratio, Taylor importance, label-loss retraining)	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured

ID	Configuration	Val acc (%)	Test acc (%)	Macro-F1	|Param|	Size (MB)	MACs	Latency (ms)
M3	KD without GA pruning (small hand-designed student, same parameter budget as M5)	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured
M4	GA-based structured pruning with retraining (label loss only)	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured
M5	GA-based structured pruning with retraining and KD (RKD)	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured	To be measured

Val acc = validation accuracy (5,000 images). Test accuracy, macro-F1 and latency were not printed in the supplied log. Size and MACs for M1 are derived from the printed architecture (Section 5.2).

Comparing M2 with M4 would show the value of the GA, M4 with M5 the value of KD, and M3 with M5 whether pruning a large network beats training a small one with KD. No result exists yet for M2 to M5, and none is claimed.

6.4 Theoretical Interpretation of Results

6.4.1 Percentage Reduction versus Percentage-Point Change

A **percentage reduction** compares a resource before and after pruning, relative to the original value. A **percentage-point change** is the plain difference between two percentages, such as two accuracies:

$$Reduction\ (\%) = \frac{X_{orig} - X_{pruned}}{X_{orig}} \times 100 \tag{21}$$

$$\Delta_{pp} = Acc_{orig} - Acc_{pruned} \tag{22}$$

For classifier D, Equation (22) gives 92.24 − 90.42 = 1.82 percentage points. In relative terms this is 1.82 / 92.24 ≈ 1.97% of the original accuracy (*analysis*), so the abstract's "1.82% decrease" is a percentage-point figure. Equation (21) gives a time reduction of (3.6 − 2.1) / 3.6 = 41.67%, a speed-up of about 1.71×. The pruned classifier keeps about 9% of the original parameters (2.81 / 31.34 ≈ 8.97%). Recomputing from the rounded table entries reproduces the reported percentages to within about 0.1 point, except classifier A's |Param| (recomputed 90.60% against 90.22% reported), because 0.11 M is a heavily rounded value.

6.4.2 Accuracy–Efficiency Trade-off

The smaller classifiers lose much more accuracy than the larger ones (7.97 points for A against 1.82 for D), so the trade-off depends on the size of the original model. A likely reason is that small models have less redundancy, but the paper does not test this (*analysis*). The same lesson applies to the notebook's baseline, which has only 0.62 M parameters: pruning it by 90% would be a much harsher test than pruning the 31.34 M-parameter classifier D, and the accuracy loss should be judged against the 79.44% baseline in percentage points.

6.4.3 Fewer FLOPs Do Not Guarantee Proportional Speed

For classifier D, FLOPs fell 90.96% but time fell only 41.67%. Memory traffic, layer overheads, small layers that under-use the processor and the measuring setup can all limit the speed-up; the paper does not explain the gap (*analysis*). This is why the plan measures latency directly instead of relying on FLOPs.

6.4.4 Observations on the Baseline

The baseline's final training accuracy (91.19%) is 11.75 percentage points above its validation accuracy (79.44%), which indicates over-fitting, and validation accuracy was still rising at the last epoch (*analysis*). Data augmentation or a longer schedule would probably improve the teacher, and a stronger teacher is useful for distillation. For the comparison to be fair, however, every model in Table 6.5 must be trained with the same recipe, so any change to the baseline recipe should be made once, before M2 to M5 are run.

6.5 Critical Evaluation

Strengths. The method targets structured pruning, so savings can show up as real speed-up on ordinary hardware. The chromosome is compact (one gene per layer), which keeps the GA tractable for models with tens of millions of parameters. The fitness function weighs accuracy, size and computation together, and RKD reached comparable accuracy in 25% fewer epochs than LRW (Paper).

Cost and dependence. Each candidate needs a fitness evaluation, so the search costs roughly population size × generations evaluations (*analysis*). The paper reports 10 to 20 generations but not the population size, the search time, or whether candidates are retrained before scoring. RKD needs up to 150 epochs of retraining and a pass through the teacher at each step, and the student inherits the teacher's mistakes (*analysis*). Results cover three networks on three datasets; other tasks and architectures are untested.

Inconsistencies and comparison limits. The text of Sec. 4.1 of [1] says the pruned model keeps more than 90.00% accuracy, but Table 1 shows 77.12% and 89.01% for classifiers A and B. The comparison with [15] and [16] uses different original accuracies (for example 92.89% against 92.63% for ResNet-32 on CIFAR-10) and does not report their parameter reductions, so it is not fully controlled. No standard deviations or repeated runs are reported.

Missing ablation. Table 6.3 compares RKD with LRW, which differ in both the loss and the learning-rate schedule. The paper does not compare GA-found ratios with uniform or random ratios, so the benefit of the GA itself, and the separate effect of distillation, are not isolated (*analysis*). The planned models M2 to M5 are designed to answer exactly this.

Reproducibility. Reproducing the work is hard because the paper omits population size, selection and mutation settings, the FLOP-counting convention, the log base and accuracy scale in the fitness, training details for R(2+1)D-18, and data splits (Table 4.2 and Section 3.1.1).

6.6 Applications, Advantages and Limitations

A smaller, faster CNN is useful wherever compute, memory or power is limited: mobile vision, embedded systems, AR/VR gesture recognition (the setting evaluated in [1]), edge AI cameras and sensors, and robots or drones. Only the gesture and CIFAR experiments were run in [1]; the method has not been validated for every application listed here.

Table 6.6: Advantages and limitations of the hybrid approach

Advantages	Limitations
Structured pruning gives real speed-up on standard hardware (Paper)	GA search needs many fitness evaluations; its cost is not reported (*analysis*)
More than 90% fewer parameters and FLOPs for the gesture model, with a 1.82-point drop for classifier D (Paper)	Accuracy loss can be large for small models (7.97 points for classifier A) (Paper, Table 1)
Compact chromosome (one gene per layer) scales to large models (Paper)	Several settings are unspecified, which limits reproducibility (Table 4.2)
RKD reached comparable accuracy in 150 instead of 200 epochs (Paper)	Needs a trained teacher and a long retraining stage (*analysis*)
Balances accuracy, size and computation in one fitness (Paper)	Fewer FLOPs do not guarantee proportional speed (Paper, Table 1)

6.7 Summary of Findings

- The reference paper reports, for classifier D of its gesture model, a 91.02% parameter reduction and a 90.96% FLOP reduction for a 1.82 percentage-point accuracy drop, and a drop in inference time from 3.6 ms to 2.1 ms (Paper).
- RKD reached accuracy close to LRW in 150 instead of 200 epochs on ResNets (Paper).
- The student's baseline CNN has 620,810 parameters and about 10.85 M MACs, and reached a best validation accuracy of 79.44% after eight epochs, with signs of over-fitting (Notebook; *analysis*).
- GA search, channel pruning and distillation on CIFAR-10 have not yet been run, so no compression result is claimed for them.

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

7. Conclusion and Future Scope

7.1 Conclusion

Deep CNNs are accurate but costly to store and run, which makes them hard to deploy on constrained devices. The reference paper [1] proposes a hybrid solution: a Genetic Algorithm searches one pruning ratio per layer, the chosen channels are removed, and the pruned network is retrained with Knowledge Distillation from the original model. The evolutionary search replaces manual tuning of layer-wise ratios with a population-based search guided by a fitness that weighs accuracy, size and operations, and distillation lets the pruned network learn from the teacher's soft outputs with a temperature that falls from 20 to 1.

The paper reports that its largest gesture classifier lost 91.02% of its parameters and 90.96% of its FLOPs for a 1.82 percentage-point accuracy drop, with inference time falling from 3.6 ms to 2.1 ms, and that RKD matched LRW-level accuracy on ResNets in 150 rather than 200 epochs. In this study, the first experimental step is complete: a baseline CNN with 620,810 parameters was trained on CIFAR-10 and reached 79.44% validation accuracy. This baseline is the teacher for the planned comparison.

7.2 Limitations

- The GA, pruning and distillation stages have not been run on CIFAR-10, so the study has no compression results of its own yet.
- The baseline was evaluated on the validation split only; test accuracy, macro-F1 and latency were not in the supplied log.
- The baseline shows over-fitting and was trained with a short schedule and no augmentation.
- The reference paper omits several settings and has no ablation separating the GA from distillation.

7.3 Roadmap and Future Scope

Table 7.1: Next steps for completing the experiment

Step	Work	Status
1	Train the baseline CNN (teacher) and save the best checkpoint	Done (Notebook)
2	Compute channel-importance scores (activation and gradient hooks)	Started; output not in the supplied log
3	Evaluate the baseline on the test set; record macro-F1, size, MACs and latency	To be done
4	Implement the GA (chromosome, operators, fitness) and run the search	To be done
5	Prune with P_{best}; retrain with and without KD (M2 to M5)	To be done
6	Run 3 to 5 seeds and fill the comparison table	To be done

Beyond this plan, future work could add an ablation against uniform or random ratios, report the search cost, test other architectures and tasks, measure latency on a real target device, and combine pruning with quantisation.

8. Appendix I: Notebook Summary, Glossary and Verification

8.1 Notebook Cells and Status

Cell	Purpose	Result printed (Notebook)
1	Upgrade torch, torchvision, scikit-learn, matplotlib, pandas	torch 2.14.1, torchvision 0.29.1, scikit-learn 1.9.1, matplotlib 3.11.2, pandas 3.1.0; dependency warnings for google-colab and cudf
2–3	Check PyTorch version and GPU	CUDA available; Tesla T4, 15.64 GB
4	Load and split CIFAR-10	45,000 train, 5,000 validation, 10,000 test; ten classes
5–6	Select device; build the CNN	cuda; 620,810 parameters
7	Train for 8 epochs and save the best checkpoint	Best validation accuracy 79.44%
8	Register hooks for channel importance	Log ends inside this cell

8.2 Glossary

Term	Meaning
Channel / filter	One feature map of a convolutional layer / the weights that produce it
Structured pruning	Removing whole channels or filters, so the layer becomes smaller
Unstructured pruning	Setting individual weights to zero, leaving the tensor shape unchanged
Chromosome / gene	A candidate solution / one element of it (here, one layer's pruning ratio)
Fitness	Score that tells the GA how good a candidate is
Crossover / mutation	Mixing two parents / randomly changing genes
Teacher / student	The original, larger model / the smaller pruned model that is trained
Logit	Raw output of the last layer before softmax
Temperature (T)	Value that softens (T > 1) the softmax output
KL divergence	Measure of how different two probability distributions are
FLOP / MAC	Floating-point operation / multiply–accumulate operation
RKD	Retraining with Knowledge Distillation, the retraining method of [1]
LRW / FT	Learning rate rewinding / fine-tuning, label-only retraining methods [6]

8.3 Numerical Verification Log

Item checked	Location in [1]	Result
Classifier D: 92.24 → 90.42 %, drop 1.82	Table 1; Sec. 4.1; abstract	Matches
Classifier D: 31.34 → 2.81 M (91.02%); 40.52 → 3.66 G (90.96%)	Table 1; abstract	Matches
Classifier D: 3.6 → 2.1 ms	Table 1	Matches (41.67%)
Classifiers A–C values	Table 1	Matches; drops re-computed
ResNet results and RKD vs LRW	Tables 2 and 3	Matches; drops re-computed

Item checked	Location in [1]	Result
κ, λ, α, β, γ, δ, T_0, E, noise range, generations	Sec. 3.1, 4	Matches
“More than 90.00% accuracy after pruning”	Sec. 4.1	Inconsistent with Table 1 (A: 77.12%, B: 89.01%)
Third fitness coefficient printed as “r”	Sec. 3.1.4, EQ 10	Read as γ (typographical inconsistency)
Baseline parameter total 620,810	Notebook (printed)	Layer sums in Table 5.1 add up

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

9. References

[1] K.-L. Chou, C.-L. Wang, Y.-C. Chen, W. Tang, Y.-T. Li, S.-C. Chang, and C.-Y. Wang, "CNN model optimization using a hybrid approach of genetic algorithm-based pruning and retraining with knowledge distillation," in *Proc. Great Lakes Symp. VLSI (GLSVLSI '25)*, New Orleans, LA, USA, Jun. 30–Jul. 2, 2025, pp. 667–672, doi: 10.1145/3716368.3735193.

[2] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[3] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. NeurIPS*, 2015.

[4] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. ICCV*, 2017, pp. 2736–2744.

[5] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," arXiv:1611.06440, 2016.

[6] A. Renda, J. Frankle, and M. Carbin, "Comparing rewinding and fine-tuning in neural network pruning," in *Proc. ICLR*, 2020.

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] G. Syswerda, "Uniform crossover in genetic algorithms," in *Proc. Int. Conf. Genetic Algorithms (ICGA)*, vol. 3, 1989.

[9] O. Köksoy and T. Yalcinoz, "Robust design using Pareto type optimization: A genetic algorithm with arithmetic crossover," *Comput. Ind. Eng.*, vol. 55, no. 1, pp. 208–218, 2008.

[10] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[12] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, "A closer look at spatiotemporal convolutions for action recognition," in *Proc. CVPR*, 2018, pp. 6450–6459.

[13] Y. Zhang, C. Cao, J. Cheng, and H. Lu, "EgoGesture: A new dataset and benchmark for egocentric hand gesture recognition," *IEEE Trans. Multimedia*, vol. 20, no. 5, pp. 1038–1050, 2018.

[14] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.

[15] H. Cheng, Z. Wang, L. Ma, X. Liu, and Z. Wei, "Multi-task pruning via filter index sharing: A many-objective optimization approach," *Cogn. Comput.*, vol. 13, no. 4, pp. 1070–1084, 2021.

[16] H. Cheng, Z. Wang, L. Ma, Z. Wei, F. E. Alsaadi, and X. Liu, "Differentiable channel pruning guided via attention mechanism: A novel neural network pruning approach," *Complex Intell. Syst.*, vol. 9, no. 5, pp. 5611–5624, 2023.

[17] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

Reference [1] is the primary source and was read in full for this report. References [2]–[16] are works cited by [1] and are listed as secondary sources for background; [17] is cited for the implementation. Venue details of [11] and [12] follow the original publications.